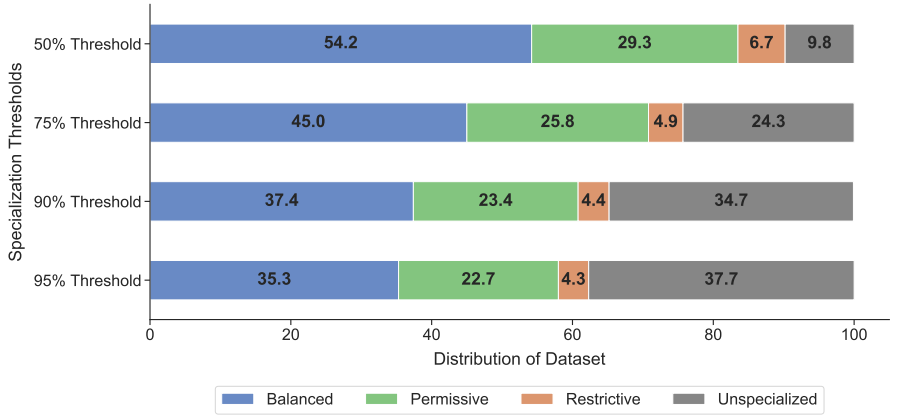


Fig. 2. Impact of specialization threshold on class distribution

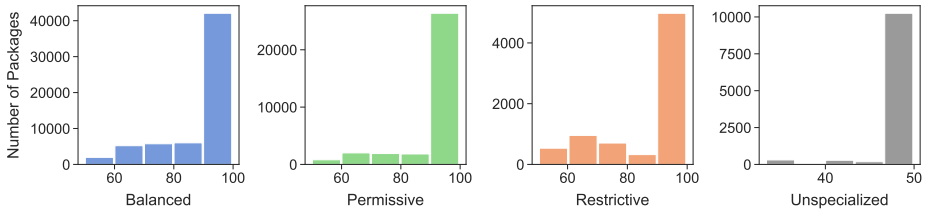


Fig. 3. Distribution of dependent agreement percentage for packages in each class

3.2 Feature selection and extraction

In this section, we explain the rationale for selecting the package features. We then explain our feature extraction procedure and the necessary pre-processing of the features.

Feature selection rationale: In order to train a suitable model in predicting dependency update strategies, we first need to select appropriate features that can capture developer needs in choosing the correct strategy. The libraries.io dataset consists of over 50 characteristics for each package, although some are highly correlated. We use the term package features to refer to characteristics from both the package on npm and its project repository. In order to determine what features in our dataset are relevant and what other features might be needed, we studied the literature to identify which package characteristics are associated with the characteristics involved in choosing and managing dependencies.

Table 1 presents each of these features. All of the studies referenced in the table are comprised of developer surveys and interviews regarding practitioner needs and practices (see Section 6). The features listed here are deemed relevant in the literature in choosing and managing dependencies, but ours is the first study to investigate their influence on the dependency update strategy. According to the reviewed literature, developers use the following characteristic groups to select dependencies:

- **Package maturity and popularity** is a recurrent factor in the literature. Prominent projects that are established in the community are a priority in selecting dependencies [4, 21, 25, 36]. Characteristics such as Age, Dependent Count, Repository Stars and Forks Count along with Repository size and Contributors count can be used as indicators for established package

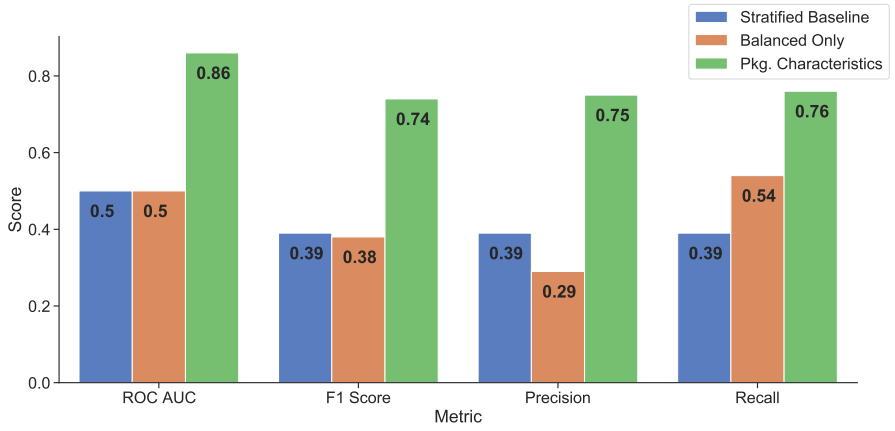


Fig. 5. Performance evaluation results

Table 3. Comparing model performance across different specialization thresholds

Threshold	Model	ROC AUC	Min. Increase	F-1 Score	Min. Increase
50%	Stratified Baseline	0.50	-	0.39	-
	Balanced Only	0.50	-	0.38	-
	Package Characteristics	0.86	72%	0.74	90%
75%	Stratified Baseline	0.50	-	0.33	-
	Balanced Only	0.50	-	0.28	-
	Package Characteristics	0.85	70%	0.67	103%
90%	Stratified Baseline	0.50	-	0.32	-
	Balanced Only	0.50	-	0.20	-
	Package Characteristics	0.86	72%	0.68	113%
95%	Stratified Baseline	0.50	-	0.32	-
	Balanced Only	0.50	-	0.18	-
	Package Characteristics	0.88	76%	0.70	119%

number of unspecialized packages for which there is no majority agreement on the update strategy. Our objective is to model the relationship between package characteristics and the common update strategy of its dependents in the npm ecosystem. A model that assumes a strictly high level of agreement among the dependents will be of limited use in practice as such agreement does not exist for many npm packages.

Finding #1: The quality of our classification model shows that package characteristics can be used as indicators of the common update strategy chosen by the package's dependent community.

Finding #2: While the balanced update strategy is recommended by npm, the recommended update strategy from the package characteristics model is better aligned with the update strategy selected by npm developers.